OUTPUT ANALYSIS

Water Depot Network System • Graph (Adjacency List), Priority Queue and Dijkstra's Algorithm in C++

1. Time Complexity Analysis

Let **V** be the number of locations (water depot, road junctions and repair locations) and **E** be the number of roads currently stored in the graph.

Operation	Time Complexity	Reason
Add Location	**O(log V)**	Duplicate check and insertion in the name map take O(log V); pushing into the names, type and adjacency arrays is O(1) amortised.
Add Road	**O(V)**	Both locations are found by number, name or partial name (worst case a scan of all V locations); the two-way edge is then added in O(1).
Display Locations	**O(V)**	Each location is printed once with its type.
Display Network	**O(V + E)**	Visits every location and every road in the adjacency list once.
Dijkstra (Shortest Route)	**O((V + E) log V)**	Each location is finalised once; each road may push one entry into the min-priority queue, and every push or pop costs O(log V).
Route Reconstruction	**O(V)**	Traces parent[] from the repair location back to the source and reverses it; the path has at most V locations.
Change Source (Modification A)	**O((V + E) log V)**	Dijkstra is run again from the new source; the distances to all locations are then displayed in O(V).

2. Space Complexity Analysis

Location data (name, type) is stored in vectors and a map, and roads are stored as (neighbour, distance) pairs in an adjacency list. Each road is saved twice because roads are two-way.

Component	Space Complexity	Reason
Total Space Complexity	**O(V + E)**	The graph stores V names and types plus 2E adjacency entries (a road is saved in both directions).
Adjacency List	**O(V + E)**	One list per location; each two-way road adds one entry to each end, so 2E entries in total.
Dijkstra Arrays	**O(V)**	dist[], parent[] and visited[] each hold one value per location.
Priority Queue	**O(E)**	Lazy insertion can leave up to one entry per road in the queue at the same time.
Auxiliary Space per Operation	**O(V)**	Route display needs a path array of at most V locations; add, search and display operations use only O(1) extra variables.

Interpretation: Adding and displaying operations only touch the data structure once or scan it linearly, so they are cheap. The main cost is Dijkstra's algorithm at O((V + E) log V), because the min-priority queue picks the nearest unvisited location in O(log V); this is faster than scanning an array for the minimum, which costs $O(V^2)$. Changing the source vertex (Modification A) repeats the same algorithm, so every new source costs the same. Because all road distances are non-negative, the result is always the true least-cost route, and the adjacency list keeps memory linear at O(V + E).